Algorithmes et programmation distribuée

Alexandrina KORNEVA

alexandrina.korneva@universite-paris-saclay.fr

Planning + Évaluation

Contenu des TP

> Threads
> Sockets
> Consensus

Évaluation

> Contrôle continu + TP noté

Sockets (prises)

Architecture Client / Serveur

Serveur (celui qui écoute)

Un serveur écoute sur un port

- Ouvre un port sur sa propre machine (ex. 5000)
- Utilise socket.bind() pour réserver le port
- Utilise socket.listen() pour attendre des appels
- Utilise socket.accept() pour accepter un appel

Client (celui qui appelle)

Un client se connecte à une adresse et un port

- Connaît l'adresse du serveur (ex. 127.0.0.1)
- Connaît le port du serveur (ex. 5000)
- Utilise socket.connect()
- Utilise socket.sendall() pour envoyer des données

Les octets

Les Sockets ne comprennent pas les listes, les dictionnaires, ou les objets Python. Ils ne comprennent que les **octets (bytes)**. Il faut *traduire* les données.

1. Objet Python (ce que nous voulons envoyer)

```python
msg = {
    'sender': 'Node A',
    'data': 'Hello!'
}
```

2. Sérialisation (JSON) et encodage en bytes

```python
json_str = json.dumps(msg)
# "{\"sender\": \"Node A\"...}"

data = json_str.encode('utf-8')
# b'{\\"sender\\": \\"Node A\\"...}'
```

3. Désérialisation à la réception

```python
# D'abord en string
json_str = data.decode('utf-8')
# Puis en objet Python
msg = json.loads(json_str)
```

Les octets

Les Sockets ne comprennent pas les listes, les dictionnaires, ou les objets Python. Ils ne comprennent que les **octets (bytes)**. Il faut *traduire* les données.

1. Objet Python (ce que nous voulons envoyer)

```python
msg = {
    'sender': 'Node A',
    'data': 'Hello!'
}
```

2. Sérialisation (JSON) et encodage en bytes

```python
json_str = json.dumps(msg)
# "{\"sender\": \"Node A\"...}"

data = json_str.encode('utf-8')
# b'{\\"sender\\": \\"Node A\\"...}'
```

3. Désérialisation à la réception

```python
# D'abord en string
json_str = data.decode('utf-8')
# Puis en objet Python
msg = json.loads(json_str)
```

C'est le pattern

Encode/Decode

&

Serialize/Deserialize.

Serveur Simple + Client Simple

```python
import socket, json

HOST = '127.0.0.1'
PORT = 8000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print("Serveur: en écoute...")

    conn, addr = s.accept() # Blocant

    with conn:
        print(f"Connecté par {addr}")
        data = conn.recv(1024) # Recevoir des octets

        # Le pattern de décodage
        msg_str = data.decode('utf-8')
        msg_obj = json.loads(msg_str)

        print(f"Reçu: {msg_obj}")
```

```python
import socket, json, time

HOST = '127.0.0.1' # Le même serveur
PORT = 8000

# L'objet à envoyer
my_message = {
  "sender": "Client",
  "message": "Hello",
  "timestamp": time.time()
}

# Le pattern d'encodage
msg_str = json.dumps(my_message)
msg_bytes = msg_str.encode('utf-8')

try:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        s.sendall(msg_bytes)
        print("Client: Message envoyé")
except ConnectionRefusedError:
    print("Client: Échec. Serveur offline?")
```

Le Peer-to-Peer (P2P)

Notre serveur est **bloquant**. Il ne peut gérer qu'un seul client, puis il s'arrête. Notre client ne fait qu'envoyer, il ne peut pas **recevoir**.

Comment être un **Serveur** (écouter les messages) et un **Client** (envoyer des messages) en même temps ?

Le Peer-to-Peer (P2P)

Notre serveur est **bloquant**. Il ne peut gérer qu'un seul client, puis il s'arrête. Notre client ne fait qu'envoyer, il ne peut pas **recevoir**.

Comment être un **Serveur** (écouter les messages) et un **Client** (envoyer des messages) en même temps ?

→ **Threads**

Le Peer-to-Peer (P2P)

Chaque nœud va exécuter deux boucles

Thread client : gère l'envoi de messages

```
while True:
    # 1. Attendre l'input de l'utilisateur
    text = input("> ")

    # 2. Sérialiser le message
    msg_bytes = json.dumps({"txt": text}).encode()

    # 3. Se connecter au pair
    with socket.socket(...) as s:
        s.connect((PEER_HOST, PEER_PORT))
        # 4. Envoyer le message
        s.sendall(msg_bytes)
```

Le Peer-to-Peer (P2P)

Chaque nœud va exécuter deux boucles

Thread serveur : tourne en arrière-plan pour écouter les messages entrants.

```python
def server_loop():
    with socket.socket(...) as s:
        s.bind((MY_HOST, MY_PORT))
        s.listen()

        while True:
            # 1. Attendre une connexion
            conn, addr = s.accept() # Bloquant!

            # 2. Gérer la connexion
            with conn:
                data = conn.recv(1024)
                msg = json.loads(data.decode())
                print(f"\nReçu: {msg}")
```

TP

TP: Un Explorateur de Fichiers

Objectif du TP: Nœud P2P

https://shorturl.at/fqqGe

Créer un script Python (p2p_chat.py) qui peut agir comme l'un des deux nœuds d'un chat P2P.

Le script prendra **deux** arguments : son propre port et le port de son pair.

```
# Terminal 1 (nœud A)
$ python3 p2p_chat.py 8000 8001

# Terminal 2 (nœud B)
$ python3 p2p_chat.py 8001 8000
```

Etapes:

1. Importer socket, threading, json, sys
2. Récupérer « my_port » et « peer_port » (sys.argv)
3. Fonction server_loop(my_port) doit créer un socket serveur (bind, listen), lancer une boucle infinie pour accepter des connexions (while True), gerer chaque connexion (recv, decode, loads, print)
4. La fonction main doit: créer et démarrer le thread serveur, récupérer le message de l'utilisateur (input), gérer l'envoi (dumps, encode, connect, sendall)

HINTS:

- `daemon=True` **force le thread a s'arreter quand le programme pricipal se termine (lors de la creation)**
- Créer un nouveau socket client pour *chaque* message que vous envoyer.
- Gérer les erreurs avec try…except

https://shorturl.at/fqqGe

Objectif du TP: Nœud P2P

```
alexandrina@alexandrinas-MacBook-Pro cours2 % python3 solution.py 800
1 8000
--- P2P Chat Node ---
Starting node. My Port: 8001, Peer's Port: 8000
Server thread started. You can now send messages.
Type your message and press Enter. Type 'exit' to quit.
> [Server Thread]: Listening for connections on port 8001...
Bonjour 1
[Message Received]: bonjour 2
>
```

```
alexandrina@mpu-174119 cours2 % python3 solution.py 8000 8001
--- P2P Chat Node ---
Starting node. My Port: 8000, Peer's Port: 8001
Server thread started. You can now send messages.
Type your message and press Enter. Type 'exit' to quit.
> [Server Thread]: Listening for connections on port 8000...
[Message Received]: Bonjour 1
> bonjour 2
>
```

TP: extensions

https://shorturl.at/fqqGe

1. Rajouter des timestamps aux messages : [Message Received][timestamp]: <message>
2. Ajouter un username en ligne de commande, l'inclure dans chaque message: [Message Received][timestamp][username]: <message>
3. Remplacer le daemon=True par un signal propre pour arrêter le serveur : taper « exit » dans le terminal
4. Au lieu de créer un nouveau socket pour chaque message, gardez une connexion ouverte vers le peer (hint: il faudra un nouveau thread sender et une queue)
5. Accepter plusieurs peers et envoyez un message à tous les peers : `python3 p2p_chat.py 8000 8001 8002 8003`